

VeriGate-FW

Verifiable AI-Gated Firmware Updates for IoT Devices
using Blockchain and IPFS

A plain-language guide to the whole project —
what it is, why it exists, and how every part works

Prepared 17 September 2026 · Software-only project · Read this before the SRS

How to read this guide

This document assumes you have **no background in cyber-security or blockchain**. Every technical idea is introduced the same way: first a real-life analogy, then the exact meaning, then how we use it in this project. Words in **bold** the first time they appear are in the glossary at the end.

1. The problem we are solving (a short story)
2. The ingredients — the ten ideas you need
3. The cast — who is involved
4. The journey of one firmware update, step by step
5. Stage 1 — the cryptographic gate (eight checks, and the attack each one stops)
6. Stage 2 — the AI risk gate
7. The policy engine — turning scores into a decision
8. The novel part — why this is research and not just engineering
9. Attack walkthroughs — watch the system catch each one
10. What we will actually build (software, folders, laptop budget)
11. Build order and timeline
12. Questions the panel will ask, with answers
13. Glossary

1. The problem we are solving

Imagine a company, **Acme**, that sells 50,000 smart door locks. Inside every lock is a small computer running a program called **firmware**. Sometimes Acme needs to change that program — to fix a bug or close a security hole. They cannot visit 50,000 homes, so they send the new firmware over the internet. This is called an **OTA update** (Over-The-Air).

Now the danger. The lock receives a file that claims to be "Acme firmware version 2.3". How does the lock know that:

- the file really came from Acme, and not from an attacker pretending to be Acme?
- the file was not **tampered** with on the way (one changed byte can open a back door)?
- the file is not an old version with a known hole, being pushed again by an attacker (a **rollback attack**)?
- Acme has not since withdrawn this version because it turned out to be faulty (**revocation**)?
- even if it is genuine Acme firmware, it does not quietly include a third-party library with a serious known vulnerability?

WHY THIS MATTERS

In 2016 the Mirai botnet took over hundreds of thousands of IoT devices. Firmware that cannot be safely updated, or that can be maliciously updated, is one of the biggest weaknesses in IoT. This is a real, current problem — regulators (EU Cyber Resilience Act, US executive orders) now require manufacturers to solve it.

Our goal, in one sentence: when a device receives a firmware update, decide — automatically, provably, and in a way anyone can audit later — whether to **INSTALL** it, **BLOCK** it, or **WAIT** for more information.

Our constraint: everything is software. The "IoT devices" are small Python programs that behave like devices. No hardware is bought or wired. This is normal in research: the security logic is identical, and it lets us test 500 devices instead of one.

2. The ingredients — the ten ideas you need

2.1 Hash (SHA-256) — the fingerprint of a file

ANALOGY

A fingerprint. Two people never share one; a tiny change in the person does not change it, but a hash is stricter: change one bit of the file and the fingerprint is completely different.

EXACT MEANING

SHA-256 is a function that takes any file and produces a fixed 32-byte number. It is one-way (you cannot rebuild the file from the hash) and collision-resistant (nobody can find two different files with the same hash). Example: `sha256("hello") = 2cf24dba...9824`.

In this project: the publisher computes the hash of the firmware file. The device recomputes it after download. If they match, the file is byte-for-byte what the publisher hashed. **Limitation:** a hash alone proves nothing about who made the file — an attacker can hash their own malware too. That is why we need the next ingredient.

2.2 Digital signature (Ed25519) — proving who wrote it

ANALOGY

A wax seal with a stamp that only Acme owns. Anyone can check the seal against Acme's public stamp-pattern; nobody else can make it.

EXACT MEANING

Ed25519 gives you a key pair: a **private key** (32 bytes, secret, held by the publisher) and a **public key** (32 bytes, published to everyone). `sign(private_key, message)` makes a 64-byte signature. `verify(public_key, message, signature)` returns true only if that exact message was signed by the matching private key. Change one bit of the message and verify fails.

In this project: the publisher signs a small document called the **manifest** (see 2.3). The device verifies the signature using the publisher's public key, which it gets from the blockchain. Now the device knows both what the file is (hash) and who vouched for it (signature).

COMMON DESIGN MISTAKE WE AVOID

Many projects sign only the hash. Then an attacker who can write to the registry can re-list an old signed firmware under a new version number: the signature still checks out, and the device happily downgrades. We sign the whole manifest — hash + version + expiry + SBOM hash together — so version and hash cannot be mixed and matched.

2.3 Manifest — the signed label on the box

A manifest is a small JSON file describing a release. Everything the device needs to make a decision is inside it, and the whole thing is signed:

```
{
  "firmwareHash": "sha256:9f2c...",    ← fingerprint of firmware.bin
  "sbomHash":     "sha256:31ab...",    ← fingerprint of sbom.json
  "version":      "2.3.0",
  "deviceModel":  "acme-lock-v2",
  "expiry":       "2027-03-01T00:00:00Z",
  "cids": { "firmware": "bafy...", "sbom": "bafy..." }, ← where to download from IPFS
  "publisherDid": "did:acme:...",
  "signature":    "ed25519:7d0e..." ← over everything above, canonicalised
}
```

Canonicalisation is a detail beginners miss: a signature is over bytes, and `{"a":1,"b":2}` and `{"b":2,"a":1}` are different bytes. We always serialise JSON with sorted keys and no whitespace before signing or verifying.

2.4 Blockchain — a notebook nobody can secretly edit

ANALOGY

A public notice-board in the town square where every notice is glued down permanently and everyone keeps a photocopy of the whole board. You can add a notice; you can never remove or alter one; and everybody can see who posted what and when.

EXACT MEANING

A blockchain is a database replicated across many computers, where entries are grouped into blocks and each block contains the hash of the previous one. Altering an old entry would change every later hash and be rejected by the other computers. We use **Ethereum**-style blockchains, which can also run small programs (see 2.5). Specifically we use **Arbitrum**, a "Layer-2" network that gives Ethereum's security at a fraction of the cost, and for daily development a **Hardhat** local node — a private one-computer blockchain that starts in two seconds and costs nothing.

What the blockchain does NOT do: it does not stop tampering. The signature does that. The blockchain gives us three things the signature cannot: (1) a public, permanent record of which firmware versions exist and which have been revoked, that the publisher cannot later deny or quietly change; (2) a place to publish public keys that does not depend on trusting one web server; (3) an audit trail of every decision the system made. You must be able to say this clearly in the viva.

2.5 Smart contract — a program that lives on the blockchain

ANALOGY

A vending machine. Rules are built in ("insert coin, press B4, get the snack"); nobody can bribe it, and everyone can read the rules.

EXACT MEANING

A smart contract is code (we write it in **Solidity**) deployed to the blockchain. It has storage (variables) and functions. Calling a function that changes storage is a **transaction**: it costs a small fee called **gas**, is signed by the caller, and is recorded forever. Reading storage is free.

In this project we write five small contracts, each a simple registry (a table with rules about who may write to it):

Contract	What it stores	Who may write
PublisherRegistry	Each publisher's public key, status (ACTIVE / REVOKED), key-rotation history, reputation score	Publisher registers itself; only the admin can revoke; reputation updated by the gateway
FirmwareRegistry	Per release: manifest hash, signature, IPFS CIDs, version, SBOM hash, expiry, revoked flag	Only an ACTIVE publisher, only for its own releases
ModelRegistry ★	Hash of each AI model version, status, successor model	Admin (the operator of the gateways)
PolicyContract ★	The weights and thresholds the policy engine uses	Admin; every change is a visible transaction

VerdictRegistry ★	Every decision the gateway made: which model, which inputs, what score, what verdict	Registered gateways only
----------------------	--	--------------------------

★ = the contracts that make this project novel (explained in section 8).

2.6 IPFS — storage where the address is the fingerprint

ANALOGY

A library where books are shelved not by title but by their fingerprint. If you ask for "book with fingerprint X" and receive a book, you can check it is the right one just by fingerprinting it. A forger cannot hand you a fake, because the fake's fingerprint would not be X.

EXACT MEANING

IPFS (InterPlanetary File System) stores files under a **CID** (Content Identifier), derived from the hash of the content. Any node holding the file can serve it; the fetcher verifies it against the CID. We run one local IPFS node (Kubo, in Docker). For evaluation runs we can use a tiny local content-addressed store that produces identical CIDs, which uses less memory.

Why not just put the firmware on the blockchain? Blockchains are extremely expensive per byte (a 1 MB firmware would cost hundreds of dollars on Ethereum). So: big files go to IPFS, and only their small fingerprints go on-chain. That is the standard pattern, and every serious paper uses it.

2.7 SBOM — the ingredients list

ANALOGY

The ingredients label on food. Firmware is not written from scratch — it contains dozens of third-party libraries (a TLS library, a JSON parser, a Wi-Fi driver...). The SBOM lists every one with its exact version.

EXACT MEANING

A Software Bill of Materials is a machine-readable list of components. We use the **CycloneDX** JSON format, generated with the free tool `syft`. Its hash goes into the manifest so it cannot be swapped.

Why we care: a library can have a publicly known security hole, called a **CVE** (Common Vulnerabilities and Exposures, e.g. [CVE-2024-3094](#)). Free databases tell you which versions are affected (**OSV.dev**), how severe it is (**CVSS** score 0–10), how likely it is to be exploited in the wild (**EPSS** probability), and whether it is actively being exploited right now (CISA's **KEV** list). By reading the SBOM and querying these databases, we can judge the risk of firmware that is perfectly genuine and untampered — something hashes and signatures can never tell you.

2.8 Machine learning model — a learned scoring function

ANALOGY

An experienced customs officer who has seen thousands of parcels and has a feel for which ones look wrong — without opening every one.

EXACT MEANING

We turn each firmware into a list of numbers (**features**: how random its bytes look, how big each section is, how much it differs from the previous version...) and train a small classical model to map those numbers to a risk score between 0 and 1. We use two scikit-learn models — a gradient-boosting regressor for SBOM risk, and an Isolation Forest for image anomalies. Both are under 1 MB and run in about a millisecond. No deep learning, no GPU.

Important honesty point: ML gives a probability-like score, not a proof. It can be wrong. That is exactly why (a) it is only Stage 2, after the strict cryptographic Stage 1, and (b) we record which model version made each decision so a bad model can be revoked (section 8).

2.9 Large language model — the explainer that never decides

An LLM (we use `qwen2.5:3b-instruct`, a small open model running locally on your laptop via Ollama) is very good at turning technical data into a readable paragraph. We give it the SBOM differences between versions, the changelog, and the top features the ML flagged, and it writes a short JSON rationale: "This release upgrades OpenSSL from 3.0.1 to 3.0.14 fixing CVE-..., but adds libcurl 7.80.0 which has a critical known vulnerability...".

DESIGN RULE

The LLM **never** makes the approve/reject decision. LLMs are non-deterministic (same input, different words) and can hallucinate. A reviewer would rightly reject a security gate that depends on one. So: the ML scores decide; the LLM only explains; its output is stored on IPFS and its CID is anchored on-chain, so the explanation of every decision is auditable too.

2.10 Merkle tree — one fingerprint for a thousand records

ANALOGY

Instead of posting 1,000 notices on the town board (expensive), post one notice with the fingerprint of the whole bundle; anyone holding one notice plus a short "path" can prove it was in the bundle.

In this project: when a gateway verifies firmware for 500 devices, it writes one Merkle root to the `VerdictRegistry` instead of 500 transactions. This is our main **efficiency** claim, and we measure the gas saved.

3. The cast — who is involved

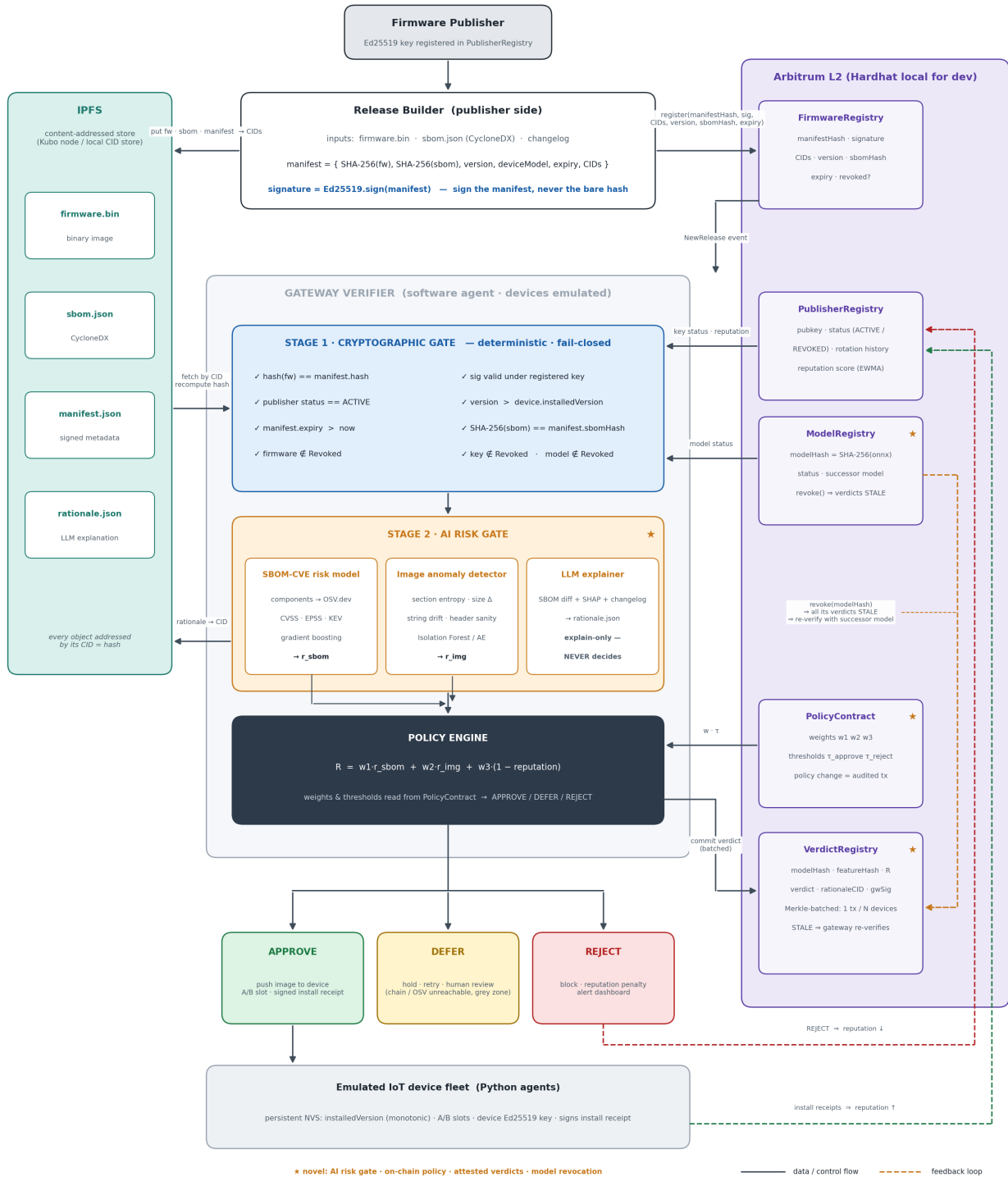
Actor	What it is in reality	What it is in our software
Firmware Publisher	The manufacturer (Acme)	A Python CLI: builds the manifest, signs it, uploads to IPFS, registers on-chain
Blockchain	Arbitrum network	Hardhat local node for development; Arbitrum Sepolia test network for final measurements (free test ETH)
IPFS	Global peer-to-peer storage	One Kubo node in Docker
Gateway Verifier	A hub/router in the home or factory that manages many devices	A Python service (FastAPI) — runs Stage 1, Stage 2, the policy engine, and writes verdicts
IoT Devices	Locks, sensors, cameras	Emulated: asyncio tasks in one process, each with its own key, stored "installed version", and A/B firmware slots
Operator / Admin	The organisation running the gateways	Deploys contracts, registers AI models, sets policy, can revoke
Dashboard	—	React web page showing releases, verdicts, device states, reputation, live logs — this is what the panel watches
Attacker	Anyone malicious	A script that launches each attack in section 9 so we can show the system catching it

4. The journey of one firmware update

Below is the full architecture. Keep it beside you while reading the steps.

VeriGate-FW — Verifiable AI-Gated Firmware Update Architecture

Software-only · blockchain (Arbitrum L2) + IPFS · emulated IoT fleet · ★ = novel component



Phase A — Publishing (done once per release)

- 1 Acme builds firmware v2.3.0** and runs `syft` to produce `sbom.json`. It writes a changelog.
- 2 Release Builder** computes `SHA-256(firmware.bin)` and `SHA-256(sbom.json)`, uploads both files to IPFS (getting two CIDs), assembles the manifest (hash, sbomHash, version, model, expiry, CIDs), canonicalises it, and **signs it with Acme's Ed25519 private key**. The manifest is also uploaded to IPFS.

- 3 **Register on-chain.** Acme sends one transaction to `FirmwareRegistry.register(...)`. The contract checks Acme's key is ACTIVE in `PublisherRegistry` and that this version is higher than Acme's previous release, then stores the record and emits a `NewRelease` event.
- 4 **Analysis at registration.** The gateway hears the event, fetches the SBOM, queries OSV/EPSS/KEV, runs the ML models once, and asks the LLM for a rationale (30 s on CPU — fine, it happens once). `rationale.json` goes to IPFS. The scores are cached for all devices.

Phase B — Verification (done for every device)

- 5 **Stage 1 — cryptographic gate.** For each device, the gateway runs the eight strict checks of section 5. Any failure ⇒ **REJECT** immediately, with the failed check named. Nothing in Stage 2 can override this.
- 6 **Stage 2 — AI risk gate.** Uses the cached scores `r_sbom` and `r_img` from step 4 plus the publisher's current reputation.
- 7 **Policy engine.** Reads weights and thresholds from `PolicyContract`, computes `R`, and outputs **APPROVE**, **DEFER** or **REJECT**.
- 8 **Verdict committed.** The gateway records {firmware, device, model hash, feature hash, R, verdict, rationale CID, its own signature}. Verdicts for a batch of devices are bundled into one Merkle root and written to `VerdictRegistry` in a single transaction.
- 9 **Action.** On APPROVE the gateway pushes the image to the device. The device (which trusts the gateway but still re-checks hash + signature itself — defence in depth) writes it to its inactive A/B slot, switches, updates its stored `installedVersion`, and signs an **install receipt**. Receipts raise Acme's reputation; REJECTs lower it.

5. Stage 1 — the cryptographic gate

These checks are **deterministic** (same input always gives the same answer) and **fail-closed** (if anything is missing or unreachable, the answer is "no"). Each one exists to stop a specific attack:

#	Check	Attack it stops	What happens without it
1	<code>SHA-256(firmware.bin) == manifest.firmwareHash</code>	Tampering in transit / on IPFS	Attacker modifies bytes of the file after signing
2	Signature valid under the publisher's registered public key	Forgery / impersonation	Attacker publishes malware "as Acme"
3	Publisher status == ACTIVE in <code>PublisherRegistry</code>	Stolen or retired key	Firmware signed with a key Acme reported stolen still installs
4	<code>manifest.version > device.installedVersion</code> (device keeps a persistent counter)	Rollback / downgrade	Attacker replays a genuine old version with a known hole
5	<code>manifest.expiry > now</code>	Freeze attack	Attacker blocks new updates forever; device never notices it is stale
6	<code>SHA-256(sbom.json) == manifest.sbomHash</code>	SBOM substitution	Attacker shows a clean ingredients list for a dirty file, fooling Stage 2
7	Firmware not revoked in <code>FirmwareRegistry</code> ; key not revoked	Known-bad release	A version Acme withdrew keeps installing
8	AI model hash not revoked in <code>ModelRegistry</code>	Compromised or buggy model ★	A verdict from a model later found faulty is still trusted

WHY THE DEVICE CHECKS TOO

The gateway does the heavy work, but each emulated device still verifies the hash and the signature itself before switching slots. If a gateway is compromised, it still cannot make a device install unsigned code. This is called **defence in depth**.

6. Stage 2 — the AI risk gate

Stage 1 answers "is this file exactly what Acme signed?" Stage 2 answers a different question: **"even if it is genuine, is it dangerous?"** Three components, run once per release:

6.1 SBOM-CVE risk model → `r_sbom`

1. Parse `sbom.json` → list of (component, version).
2. Query **OSV.dev** (free API) for each → list of CVEs.
3. For each CVE fetch **CVSS** severity (0–10), **EPSS** exploit probability (0–1), and whether it is on the **KEV** list (actively exploited).
4. Build a feature vector: max CVSS, sum of EPSS, KEV count, number of components, number of out-of-date components, average dependency age.
5. A gradient-boosting model maps this to `r_sbom` $\in [0,1]$. We train it on SBOMs generated from real open-source firmware releases (OpenWrt, ESPHome, Tasmota) with real CVE data, and we evaluate it as a ranking against the plain CVSS/EPSS/KEV formula from the literature.

6.2 Image anomaly detector → `r_img`

Looks at the firmware binary itself without running it. Features: entropy of each 1 KB chunk (mean, std, max — packed or encrypted payloads have very high entropy), ratio of printable strings, section sizes and their change from the previous version, header sanity flags, and length of any bytes appended after the declared image end. An **Isolation Forest** trained on ~200 genuine releases scores how "unusual" a new image looks. Tampered samples for evaluation are generated by us with documented mutations: byte patches, appended payloads, section swaps, packing, and downgrade relabelling.

SCOPE STATEMENT (PUT THIS IN THE PAPER)

This detector catches structural tampering. It will **not** catch a competently built malicious firmware that has normal structure. It is a layer, not a guarantee — Stage 1 plus SBOM analysis plus reputation are the other layers.

6.3 LLM explainer → `rationale.json`

Input: SBOM diff vs previous version, changelog, the top-3 SHAP feature attributions from the two models. Output (strict JSON): `{summary, top_risks[], recommended_action}`. Stored on IPFS; CID in the verdict. Read-only for humans on the dashboard and in DEFER reviews.

7. The policy engine — turning scores into a decision

$$R = w1 \cdot r_sbom + w2 \cdot r_img + w3 \cdot (1 - reputation) \quad (\text{all terms in } [0,1])$$

if $R < \tau_approve$ → APPROVE

if $R \geq \tau_reject$ → REJECT

otherwise → DEFER

Also DEFER if: blockchain unreachable · OSV lookup failed · model status unknown

`w1, w2, w3,  $\tau\_approve$ ,  $\tau\_reject$` live in `PolicyContract`. Changing them is a transaction — visible, timestamped, attributable. An operator cannot quietly loosen the policy to let something through.

Why three outcomes and not two? Real systems have grey zones and outages. A gate that must say yes or no when it cannot reach its data sources will either be unsafe (says yes) or useless (says no to everything). DEFER — hold, retry later, or escalate to a human with the LLM rationale — is what makes the design realistic.

8. The novel part — why this is research

Sections 2–7 describe a good system, but most pieces have been built before (hash + signature on-chain since 2018; SBOM on-chain in 2026). Here is what nobody has combined, and why it matters:

NOVELTY 1 — ATTESTED AI VERDICTS

Every decision is written to the chain with the **hash of the model that made it** and the **hash of the inputs it saw**. Anyone with the model file and the inputs can re-run the inference and check the score matches. The AI decision becomes reproducible and accountable — without heavy zero-knowledge machinery, which is far too slow for IoT.

NOVELTY 2 — MODEL REVOCATION

Publisher keys can be revoked. Firmware can be revoked. In every existing paper, the AI model is trusted infrastructure that can never be wrong. Here the model is **just another supply-chain artefact**: if version 3 of the anomaly detector turns out to have a blind spot, the admin revokes its hash; every verdict it produced is marked STALE; gateways automatically re-verify affected firmware with the successor model. That is the honest answer to "what if your AI is wrong?"

NOVELTY 3 — POLICY ON-CHAIN, DECISION OFF-CHAIN

Inference runs on the gateway (fast, free), but the rules it must apply are on-chain (auditable). This split gives accountability without the cost of on-chain computation.

EFFICIENCY CLAIM — MERKLE-BATCHED VERDICTS

One transaction per batch of devices instead of one per device. The nearest prior work (LedgerGuard, 2026) commits per device. We measure the gas difference.

Combined with a two-stage gate, three outcomes, SBOM-aware risk, and an explain-only LLM, this is a defensible contribution. The related-work section of the paper will cite the closest neighbours honestly: FOTB (2018), Baza et al. (2019), DIDAuth-IoTFW (2025), LedgerGuard (2026), and the SBOM-triage pipeline (arXiv 2601.01308, 2026), and show in a table which of our features each one lacks.

9. Attack walkthroughs

These are the scenarios the demo script will run in front of the panel. Each row is one attack; the "caught by" column is what the dashboard will show.

Attack	What the attacker does	Caught by	Outcome
Tampering	Flips bytes in <code>firmware.bin</code> on IPFS or in transit	Stage 1 #1 (hash mismatch)	REJECT

Forgery	Publishes own firmware signed with own key, claiming to be Acme	Stage 1 #2 (signature does not verify under Acme's registered key)	REJECT
Stolen key	Uses Acme's leaked key after Acme reported it	Stage 1 #3 (key REVOKED)	REJECT
Rollback	Replays genuine v2.1 to a device on v2.3	Stage 1 #4 (version not greater)	REJECT
Freeze	Blocks the device from seeing any new release	Stage 1 #5 (current manifest expired → alert)	DEFER + alert
SBOM swap	Pairs a clean SBOM with a firmware full of vulnerable libraries	Stage 1 #6 (SBOM hash mismatch)	REJECT
Vulnerable-but-genuine	Acme honestly ships firmware containing a library on the KEV list	Stage 2 (<code>r_sbom</code> high) → policy	REJECT or DEFER
Hidden payload	Genuine-looking release with 200 KB of packed code appended	Stage 2 (<code>r_img</code> high: entropy + appended bytes)	DEFER for review
Bad publisher history	Publisher with many recent rejections ships a borderline release	Stage 2 (reputation term)	DEFER
Poisoned model ★	Anomaly-detector v3 is found to have a blind spot after approving 300 devices	Admin revokes model hash → verdicts STALE → re-verify with v4	Affected devices re-evaluated; any now-failing ones flagged
Policy tampering ★	Rogue operator lowers thresholds	Change is an on-chain transaction — visible in the audit log	Detected, attributable
Outage	Blockchain RPC or OSV.dev unreachable	Fail-closed rule	DEFER , retry

10. What we will actually build

Component	Technology	Notes
Smart contracts (5)	Solidity, Hardhat, tests in JS/TS	Small; the hard part is getting access control right
Publisher CLI	Python: PyNaCl (Ed25519), web3.py, ipfshttpclient	<code>publish --fw firmware.bin --sbom sbom.json --version 2.3.0</code>
Gateway service	Python FastAPI, web3.py, scikit-learn, ONNX runtime, SHAP	Listens for events, runs both stages, commits verdicts
Risk analysis	OSV.dev API, EPSS CSV, KEV JSON (all free, cached locally)	Cache so the demo works offline
LLM explainer	Ollama + <code>qwen2.5:3b-instruct</code>	~2 GB; runs once per release; result cached on IPFS
Device fleet emulator	Python asyncio; each device = key + NVS JSON file + two slot files	20–50 live for the demo, 500 headless for evaluation
Storage	Kubo (IPFS) in Docker; fallback local CID store	Identical CIDs either way
Dashboard	React + Vite, ethers.js	Releases, verdicts, fleet map, reputation, live log, attack buttons
Attack scripts	Python	One command per row of section 9
Evaluation	Python notebooks → CSV → matplotlib	Latency, gas, F1, AUROC, time-to-invalidate

Proposed folder layout

verigate-fw/	
contracts/	Solidity + Hardhat tests + deploy script
publisher/	release builder CLI
gateway/	FastAPI service: stage1/, stage2/, policy/, verdicts/
ml/	feature extraction, training scripts, exported .onnx models
fleet/	device emulator
attacks/	one script per attack scenario
dashboard/	React app
evaluation/	experiment runners, results CSV, figures
docs/	this guide, SRS, paper drafts, diagrams

Laptop budget (i7-1255U, 15 GB RAM, no GPU)

Hardhat ≈ 300 MB · Kubo ≈ 250 MB · Ollama + Qwen 3B ≈ 2.5 GB · gateway + fleet ≈ 400 MB · dashboard dev server ≈ 300 MB · browser ≈ 1 GB → **about 5 GB total**. Nothing needs a GPU. Everything is free: local chain, test-network ETH from faucets, open data, open models.

11. Build order and timeline (≈14 weeks)

Weeks	Deliverable	You can demo
1-2	Contracts + tests; publisher CLI; IPFS put/get; manifest signing	Publish a release and see it on-chain
3-4	Gateway Stage 1; device emulator; A/B install; install receipts	Tampered / forged / rollback firmware rejected live
5-6	Dashboard; attack scripts; reputation; VerdictRegistry + Merkle batching	Full crypto path with audit trail — a complete baseline demo
7-8	SBOM pipeline: syft, OSV/EPSS/KEV, feature extraction, gradient-boosting model	Vulnerable-but-genuine firmware deferred/rejected
9-10	Image anomaly detector; mutation generator; LLM explainer; model registry + revocation flow	Hidden payload caught; model revocation replays verdicts
11-12	Evaluation runs on Hardhat and Arbitrum Sepolia; all figures	Numbers for the paper
13-14	Paper writing; demo rehearsal; SRS/report finalisation	Panel-ready

The order is deliberate: by week 6 you have a working, demonstrable system even if the ML takes longer than planned.

12. Questions the panel will ask

Question	Answer
Why blockchain? A signed file on a server also proves authenticity.	Correct — the signature proves authenticity. The blockchain adds what a server cannot: revocation and key records the publisher cannot silently alter, no single trusted server, and a permanent audit trail of every decision including the AI's.
Isn't blockchain too slow/expensive for IoT?	We never put firmware on-chain, only fingerprints; we use a Layer-2 (Arbitrum) and batch verdicts with a Merkle root, one transaction per batch. We report measured gas and latency.
What if the AI is wrong?	Two answers. First, it can only tighten, never loosen — Stage 1 crypto checks cannot be overridden. Second, every verdict names the model that made it, and models can be revoked, which automatically re-verifies affected firmware. No prior system has this.
Why is the LLM not making decisions?	Non-deterministic and can hallucinate. It explains; deterministic models decide. The explanation is still anchored on-chain for auditability.
Where does your training data come from?	Real SBOMs from real open-source firmware releases with real CVE data; tampered images generated by a documented mutation generator that we release. We say clearly it is not real-world malware.
Why no hardware?	The security logic is identical in emulation, and emulation lets us evaluate 500 devices and reproduce results exactly. The base paper (DIDAuth-IoTFW) used hardware; we position ours as the scalable, reproducible complement.
What does Uptane (the automotive standard) not do that you do?	Uptane has no accountability for the analysis decision, no shared revocation of an analysis model, and no cross-vendor public audit trail.
What is your single biggest limitation?	The anomaly detector detects structural tampering, not well-crafted malicious builds. We state this scope and rely on the other layers for it.

13. Glossary

Term	Meaning
A/B slots	Two firmware storage areas on a device; the new image is written to the inactive one, then the device switches. If it fails to boot, it switches back.
Arbitrum	A Layer-2 blockchain on top of Ethereum; same tools, ~100× cheaper.
CID	Content Identifier: the IPFS address of a file, derived from its hash.
CVE / CVSS / EPSS / KEV	A known vulnerability ID / its severity 0-10 / its exploit probability / the list of ones exploited in the wild.
CycloneDX	A standard JSON format for SBOMs.

Ed25519	A fast, small digital-signature algorithm.
Fail-closed	When in doubt or on error, deny.
Gas	The fee for a blockchain transaction, proportional to the work it does.
Gateway	The hub that manages many devices and does the heavy verification.
Hardhat	A developer tool that runs a private Ethereum blockchain on your laptop.
Isolation Forest	A classical ML algorithm that scores how unusual a data point is.
Manifest	The small signed JSON describing a release.
Merkle root	A single hash summarising many records, with cheap proofs of membership.
NVS	Non-Volatile Storage: the device's small persistent memory (we use a JSON file).
ONNX	A portable file format for trained ML models; we hash it to identify a model version.
OSV.dev	Google's free vulnerability database with an API.
OTA	Over-The-Air: updating a device remotely.
Revocation	Declaring something (a key, a firmware, a model) no longer trusted.
Rollback attack	Tricking a device into installing an older, vulnerable version.
SBOM	Software Bill of Materials: the list of components inside the firmware.
SHAP	A method to explain which features drove an ML score.
Smart contract	A program stored on and executed by the blockchain.
Solidity	The language smart contracts are written in.
Stale verdict	A past decision made by a model that has since been revoked; must be redone.

Prepared for the VeriGate-FW final-year project. Next document: the Software Requirements Specification (SRS).